MACHINE LEARNING FROM SCRATCH

Learning from Examples

From the perceptron to honest evaluation

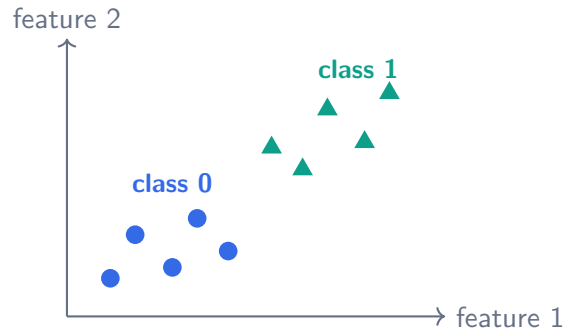
Lecture 1

The central question

A program can follow rules that we write.

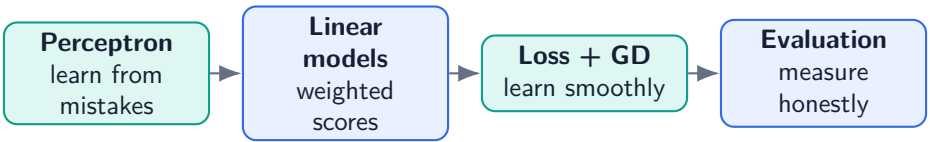
Machine learning asks something different

Can the program discover a useful rule from labeled examples?



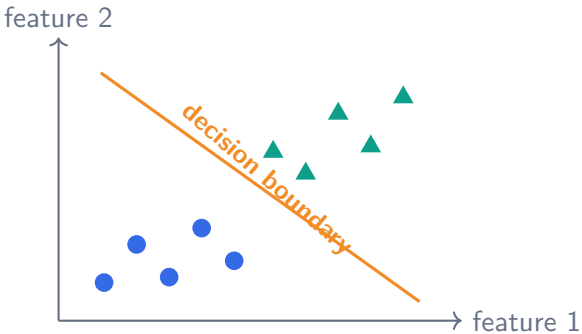
Where should a new point belong?

Today's chain of ideas



- Begin with the simplest trainable classifier
- Reuse its weighted score for regression and probabilities
- End by asking whether the learned rule will generalize

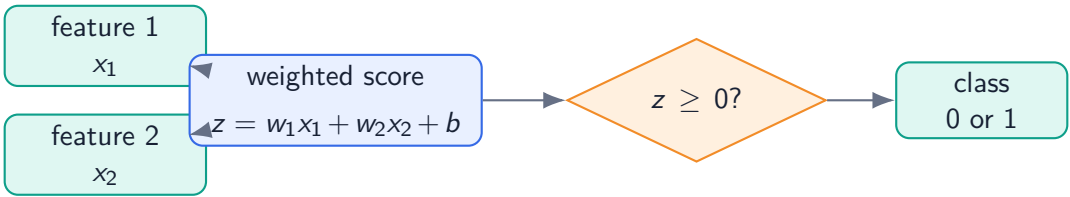
Start with a straight decision boundary



- One side predicts class 0
- The other predicts class 1
- Training moves the boundary

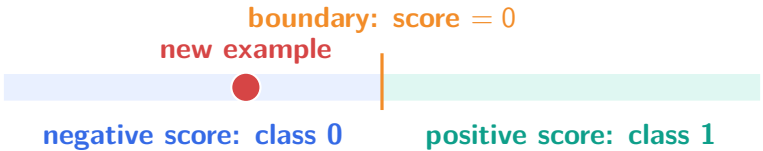
Takeaway: A perceptron is a trainable linear classifier.

The boundary comes from a weighted score



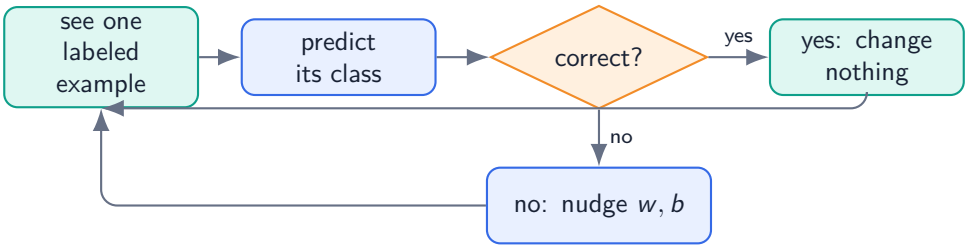
- Weights control the boundary's direction
- The bias moves the boundary without rotating it

A score becomes a hard decision



Takeaway: Prediction is easy once the weights and bias are known. Learning them is the interesting part.

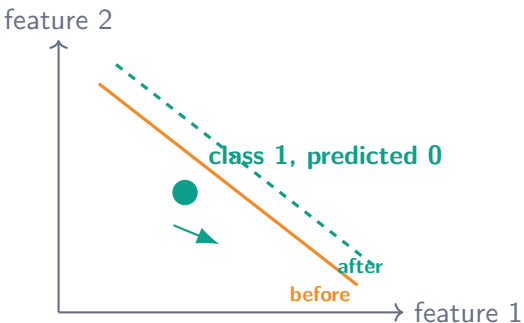
The perceptron learns from mistakes



Mistake-driven rule

Correct examples make no update. Wrong examples move the boundary toward a better decision.

What does one update try to do?



- Increase the example's score if its true class is 1
- Decrease the score if its true class is 0
- Repeat over the data for several passes

The perceptron is powerful—and limited

What it gives us

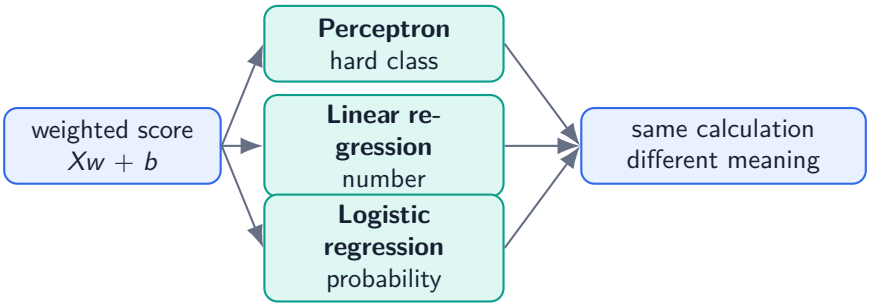
a complete learning loop
a linear boundary
an intuitive update

What it does not give

probabilities
a smooth loss
a solution for curved boundaries

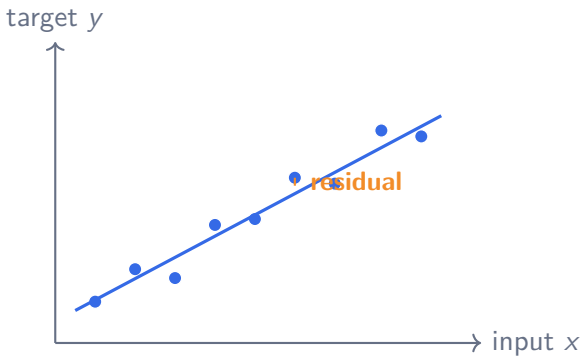
How can we keep the weighted score but obtain richer outputs and smoother learning?

One score, three interpretations



Takeaway: The output meaning determines the activation, loss, and evaluation metric.

Linear regression predicts a number



$$\hat{y} = wx + b$$

- w: slope
- b: vertical offset
- residual: prediction minus answer

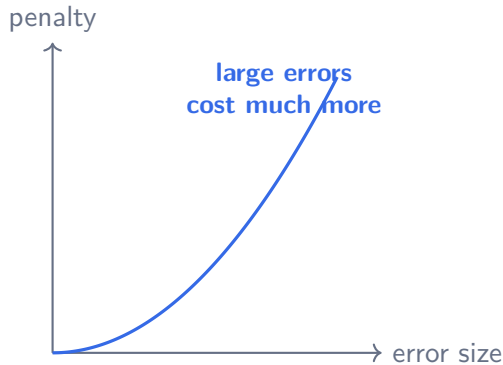
A loss defines what “better” means

- A model makes many residuals
- We need one number to compare parameter choices
- The loss becomes the training objective

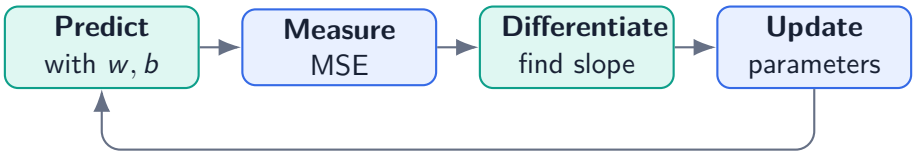
Mean-squared error (MSE)

Square each residual, then average.

$$\text{MSE} = \text{average}((\hat{y} - y)^2)$$

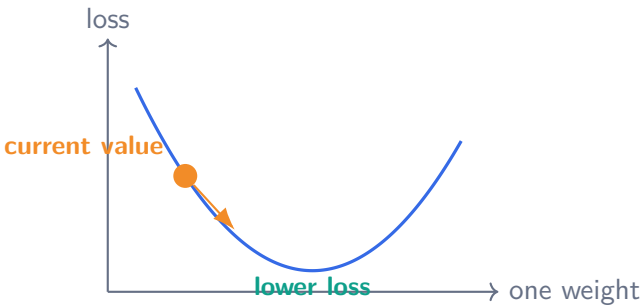


Now learning has a direction



Takeaway: Gradient descent replaces mistake-by-mistake jumps with small steps that reduce a smooth loss.

A gradient is a downhill signpost



$$\text{new value} = \text{old value} - \eta \times \text{gradient}$$

η is the **learning rate**: the step size.

The learning rate controls convergence

Too small

safe, but slow

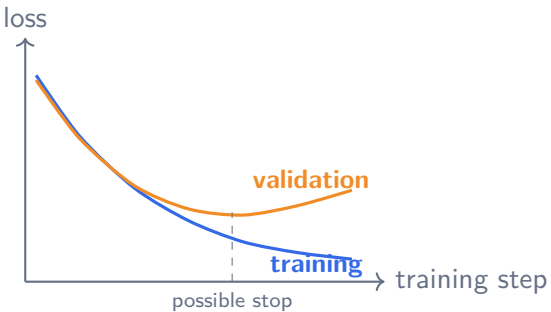
Useful

steady convergence

Too large

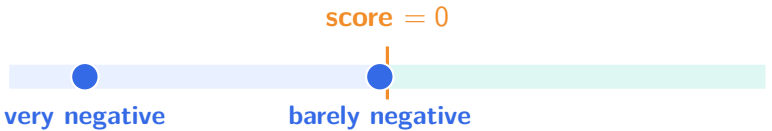
jumps or diverges

Convergence is not the same as generalization



- Training loss: fit to seen examples
- Validation loss: behavior on unseen examples
- A growing gap suggests overfitting

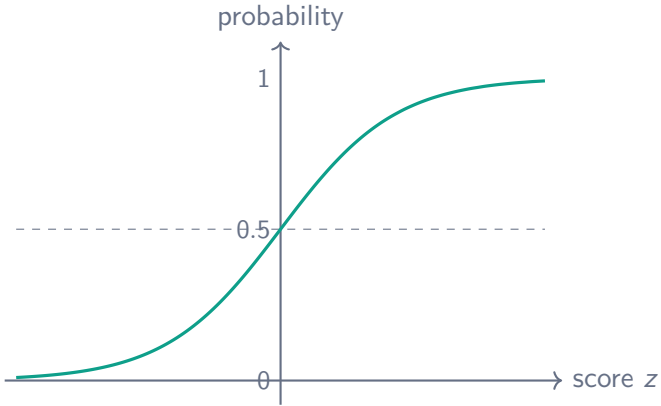
A hard score cannot express uncertainty



The perceptron calls both points class 0, but they should not express the same confidence.

Can we turn distance from the boundary into a smooth value between 0 and 1?

The sigmoid turns score into probability



$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- smooth
- bounded
- differentiable

Activation and loss solve different problems

Activation

transforms model output

SCORE → PROBABILITY

Loss

grades the prediction

PROBABILITY → PENALTY

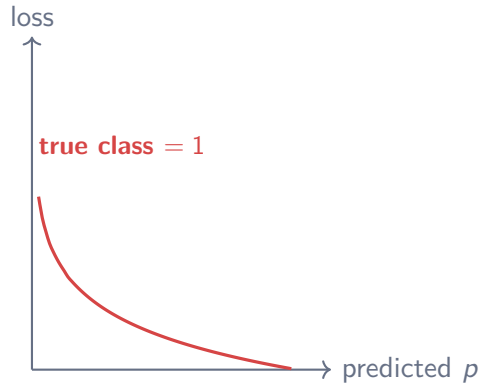
Takeaway: For logistic regression, sigmoid is the activation and binary cross-entropy is the loss.

Binary cross-entropy cares about confidence

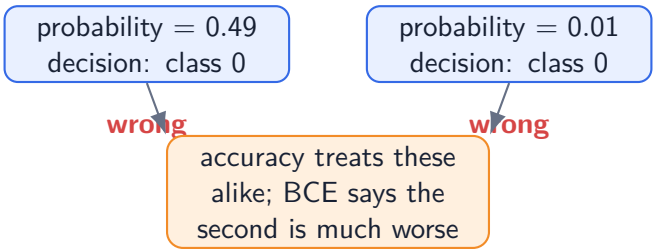
Suppose the true class is 1

Predict 0.9: small penalty
Predict 0.6: moderate penalty
Predict 0.1: very large penalty

- Correct and confident is rewarded
- Wrong and confident is punished strongly
- Smooth changes give useful gradients

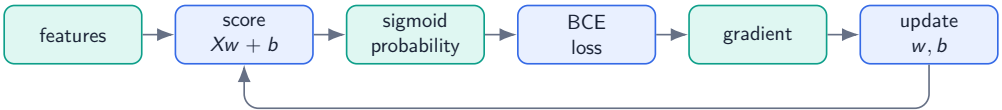


Why not use accuracy as the training loss?



Takeaway: A training loss must show how to improve before the hard decision flips.

Logistic regression reuses gradient descent



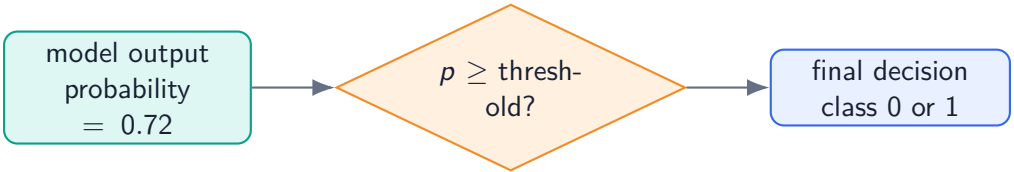
- The model learns a smooth probability, not a final action
- A separate threshold later turns probability into a decision

Perceptron and logistic regression are cousins

	Perceptron	Logistic regression
Shared core	weighted score	weighted score
Output	hard class	probability
Learning signal	mistakes	BCE gradient
Updates	only when wrong	every non-perfect prediction
Typical use	simple decisions	risk ranking / probability

Takeaway: Same linear geometry; different output and learning signal.

Probability and decision are separate



- Training learns weights that produce probabilities
- Validation chooses a threshold for the real decision
- Changing the threshold does not retrain the probability model

Every binary prediction lands in one box

		Actual: Positive	Actual: Negative
Predicted	**Positive**	**TP**: found	**FP**: false alarm
	Negative	**FN**: missed	**TN**: rejected

- Accuracy merges all four boxes into one number
- Precision and recall focus on different kinds of error

The accuracy trap

2 fraudulent

98 legitimate

Predict “legitimate” every time:

98%

accuracy

But no fraud is found.

Takeaway: Class imbalance can make a useless classifier look excellent.

Metrics answer different questions

Metric	Question	Formula
Accuracy	How often are we right?	$(TP + TN)/\text{all}$
Precision	When we flag, how often right?	$TP/(TP + FP)$
Recall	Of all positives, how many found?	$TP/(TP + FN)$
F1	Balance precision and recall	harmonic balance
F2	Favor recall more strongly	recall-weighted balance

Takeaway: Choose a metric because it matches the task, not because it is familiar.

Precision and recall express a trade-off

High precision

few false alarms

some positives may be missed

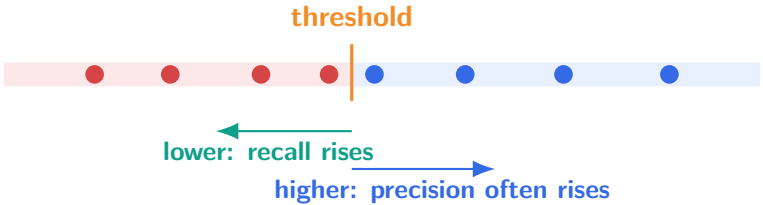
High recall

find most positives

accept more false alarms

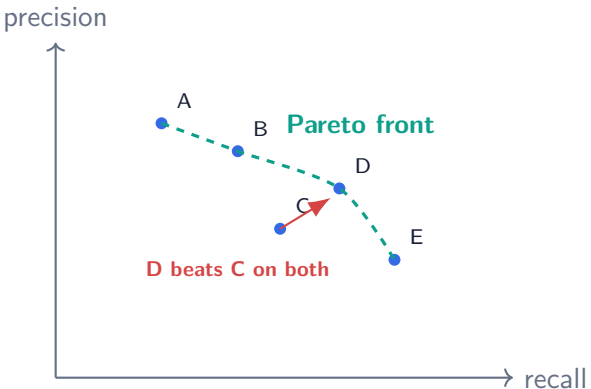
Which error matters more in the application we are building?

The threshold moves the trade-off



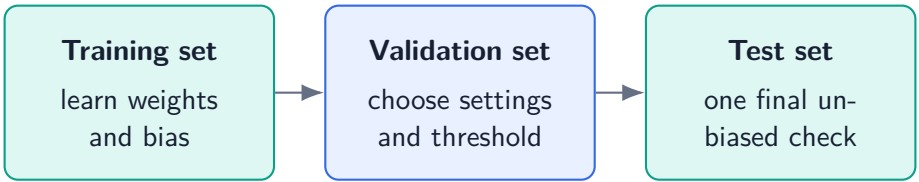
Takeaway: There is no universally correct threshold—only a threshold chosen for a purpose.

Remove models that are worse in both ways



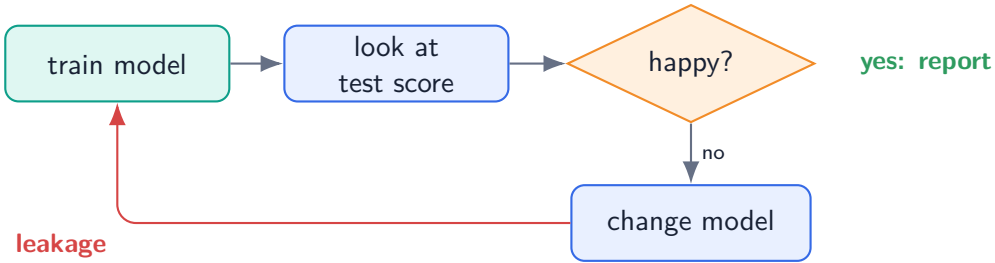
- Discard a model if another has both higher precision and higher recall
- Among the remaining models, context decides

Three datasets, three different jobs



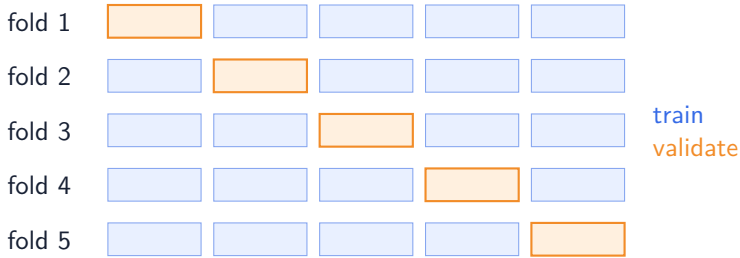
Takeaway: Every time a test result changes what we do, the test quietly becomes training data.

This loop leaks information



If we try 20 models and report the best test score, is the test still untouched?

Cross-validation makes better use of limited data

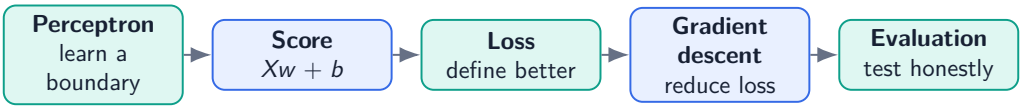


- Each fold gets one turn as validation data
- Average scores to compare model choices
- Inspect the spread to understand stability
- Keep the final test set outside the entire process

The honest workflow

- Fit preprocessing on training folds only
- Choose learning rate, features, and threshold without the test set
- Report the final metric and the cost of each kind of mistake

The logic of the lecture



- Linear and logistic regression reuse the perceptron's weighted score
- Activation determines the output; loss determines learning
- A useful model needs both good predictions and honest evaluation

Reference: model and loss formulas

$$\begin{aligned}
\text{Weighted score:} \quad & z = Xw + b \\
\text{Linear prediction:} \quad & \hat{y} = z \\
\text{Logistic probability:} \quad & p = \frac{1}{1 + e^{-z}} \\
\text{MSE:} \quad & \text{average}\big((\hat{y} - y)^2\big) \\
\text{BCE:} \quad & -\,\text{average}\big(y \log p + (1 - y) \log(1 - p)\big)
\end{aligned}$$

Reference: metric formulas

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R}$$

$$F_2 = \frac{5PR}{4P + R}$$